

Technische Hochschule Deggendorf
Fakultät Angewandte Informatik
Studiengang Bachelor Cybersecurity

LEICHTGEWICHTIGE KRYPTOGRAPHIE AUF
RESSOURCENBESCHRÄNKTEN
ARM-MIKROCONTROLLERN:
BENCHMARK-VERGLEICH VON ASCON-AEAD128
UND AES-128-GCM

LIGHTWEIGHT CRYPTOGRAPHY ON CONSTRAINED
ARM MICROCONTROLLERS:
A BENCHMARK COMPARISON OF ASCON-AEAD128
AND AES-128-GCM

Projektarbeit im Kurs *Kryptologie 2:*
Bachelor of Science (B.Sc.)
an der Technischen Hochschule Deggendorf

Vorgelegt von:
Johannes Zipper
Matrikelnummer: 22411902

Am: 29. July 2026

Prüfungsleitung:
Prof. Dr. Martin Schramm

Schaeffler Ansprechpartner:
Tobias Spieker

Abstract

Authenticated encryption is a baseline requirement for in-vehicle communication. Standards such as AUTOSAR SecOC define message authentication across the vehicle network, and ISO/SAE 21434 places such measures inside a documented cybersecurity risk management process. In practice the algorithm used is almost always AES in Galois/Counter Mode. A large share of automotive nodes, however, are low-cost 32-bit microcontrollers without a hardware cryptographic accelerator. On these parts AES-128-GCM runs entirely in software, including the GHASH field multiplication, which maps poorly onto a minimal instruction set. NIST standardized Ascon in SP 800-232 as a lightweight alternative, but quantitative performance data on representative Cortex-M hardware is scarce. This work addresses that gap.

The evaluation uses the NUCLEO-F072RB (Cortex-M0, 128 KB Flash, 16 KB SRAM) and the NUCLEO-L476RG (Cortex-M4F), with the Cortex-M4F clocked at 48 MHz to match the Cortex-M0 ceiling, so that timing differences reflect architecture rather than clock frequency. Both algorithms are measured as software-only implementations, using the official Ascon reference code and mbedTLS 3.6.2. mbedTLS is configured with the small four-bit GHASH table, which is the memory-frugal configuration of AES-GCM rather than its speed ceiling. Execution time is taken from a 32-bit free-running timer at one tick per cycle, with interrupts masked and one warmup iteration discarded. Both implementations are verified against published known-answer test vectors on target. The matrix covers message sizes from 1 to 10 000 bytes and the optimization levels -O0 through -Os.

Ascon-AEAD128 was faster than AES-128-GCM at every measured message size, on both boards, and at every optimization level. The advantage appears in the fixed per-call cost that dominates short messages and in the per-byte cost that dominates long messages, so it does not depend on the message profile of the application. The largest ratio is approximately 4.6 in favor of Ascon-AEAD128, at -O1 and -O2 on the Cortex-M0, against approximately 2.0 on the Cortex-M4F.

At -Os, the level a memory-constrained deployment would realistically choose, AES-128-GCM required approximately 3.8 times the Flash of Ascon-AEAD128 on both boards. At -O1 through -O3 the relation inverts, because the compiler unrolls the permutation, so the speed advantage and the code-size advantage are obtained together only when building for size. Within the reported build configuration the results give a quantitative basis for preferring Ascon-AEAD128 on constrained, software-only targets, with the advantage growing as the target becomes more constrained.

Declaration on the Use of AI-Based Tools

AI-based tools were used in the preparation of this thesis. The nature and extent of that use is listed below:

- **Claude (Anthropic), models Claude Opus 4.5 and Claude Opus 5:** drafting and linguistic revision of text based on my own outlines and my own measurement data, condensing of existing sections, suggestions for chapter structure, support with LaTeX formatting and figure integration, and debugging of the analysis scripts.

The conception of the work, the design of the measurement setup, the execution of all measurements, and the evaluation and interpretation of the results are my own. All text produced or revised with AI-based tools was reviewed by me for accuracy, corrected where necessary, and is my responsibility. No measurement values, results, or bibliographic references were generated by AI tools.



Deggendorf, July 29, 2026 Johannes Zipper

Contents

Abstract	ii
Declaration on the Use of AI-Based Tools	iii
1 Introduction	1
1.1 Cryptography in the Automotive Domain	1
1.2 Cryptographic Requirements of Modern ECUs	1
1.3 The Limits of AES on Low-End Microcontrollers	2
1.4 The NIST Lightweight Cryptography Standardization Process	2
1.5 Objective and Research Questions	2
1.6 Structure of This Paper	3
2 Background	4
2.1 Authenticated Encryption with Associated Data (AEAD)	4
2.2 AES-128-GCM	4
2.2.1 The AES Block Cipher	4
2.2.2 Galois/Counter Mode (GCM)	5
2.2.3 Software-Only Scope	5
2.3 The ASCON Family	7
2.3.1 The ASCON Permutation	7
2.3.2 Sponge and Duplex Construction	7
2.3.3 Ascon-AEAD128	7
2.3.4 Ascon-Hash256	8
2.3.5 Ascon-80pq and the Post-Quantum Outlook	8
2.4 Architectural Comparison: Sponge vs. Block Cipher + GCM	8
2.5 Target Platforms	9
2.5.1 ARM Cortex-M0 — NUCLEO-F072RB	9
2.5.2 ARM Cortex-M4F — NUCLEO-L476RG	9
2.6 Related Work	9
3 Methodology	10
3.1 Overview and Experimental Design	10
3.2 Target Platforms	10
3.2.1 NUCLEO-F072RB (Cortex-M0)	11
3.2.2 NUCLEO-L476RG (Cortex-M4F)	11
3.2.3 Clocking and the Matched-Frequency Decision	11
3.3 Toolchain and Build System	11

Contents

3.4	Cryptographic Implementations	12
3.4.1	AES-128-GCM (mbedTLS)	12
3.4.2	Ascon-AEAD128 (Reference Implementation)	12
3.4.3	Uniform Wrapper Interface	12
3.5	Correctness Verification	13
3.6	Measurement Methodology	13
3.7	Benchmark Parameters	14
3.7.1	Input Sizes	14
3.7.2	Optimization-Level Matrix	14
3.8	Footprint Measurement	14
3.9	Harness Validation	15
3.10	Reproducibility and Known Limitations	15
4	Results	16
4.1	Overview	16
4.2	Harness Validation	16
4.3	Execution Time	16
4.3.1	Runtime Across Message Sizes	16
4.3.2	Effect of Optimization Level	16
4.4	Throughput	17
4.5	Code Size	18
4.6	Summary of Results	19
5	Discussion	20
5.1	Performance (Research Question 1)	20
5.2	Flash Footprint (Research Question 2)	20
5.3	Optimization Sensitivity (Research Question 3)	20
5.4	Architecture Dependence (Research Question 4)	21
5.5	The Fairness of the Comparison	22
5.6	Practical Conclusions	22
6	Conclusion and Outlook	23
6.1	Conclusion	23
6.2	Outlook	23

List of Figures

2.1	AES-128-GCM mode of operation	6
2.2	Ascon-AEAD128 mode of operation	8
4.1	Median execution time vs. message size at -02 on both boards (logarithmic axes). Ascon-AEAD128 is consistently faster than AES-128-GCM at every size.	17
4.2	Median cycles per byte vs. message size at -02 on both boards (logarithmic axes). The asymptotic value at large sizes reflects the steady-state per-byte encryption rate.	18
4.3	Marginal Flash cost at -0s for each algorithm and board. Ascon-AEAD128 requires approximately $3.8\times$ less Flash than AES-128-GCM at this level. . . .	18

List of Tables

3.1	Target platform properties	10
4.1	Median execution time (cycles) and ratio at 1000 bytes per optimization level. .	17
4.2	Marginal Flash cost (bytes) and ratio per optimization level.	19

1 Introduction

1.1 Cryptography in the Automotive Domain

Modern vehicles are no longer purely mechanical systems. Today's cars contain upwards of hundreds of Electronic Control Units (ECUs) responsible for functions ranging from engine management and brake control to infotainment, over-the-air software updates, and vehicle-to-everything communication. These systems are increasingly interconnected, over internal buses such as CAN, LIN, and Automotive Ethernet and external interfaces such as V2X and telematics, which increases the attack surface exposed to adversaries. A successful intrusion into a safety-relevant ECU carries consequences that extend well beyond data loss: it can compromise brake controllers, manipulate steering actuators, or falsify sensor data, directly endangering human lives.

Cryptographic protection of ECU communication is therefore not optional. Standards such as AUTOSAR SecOC (Secure Onboard Communication) and ISO/SAE 21434 establish requirements for message authentication and confidentiality across the vehicle network. In practice, the algorithm most commonly used to fulfill these requirements is the Advanced Encryption Standard (AES). It has served as the global symmetric encryption standard since its NIST standardization in 2001 [1] and is deeply embedded in automotive security stacks, ranging from hardware security modules (HSMs) to communication middleware.

1.2 Cryptographic Requirements of Modern ECUs

ECUs span a wide spectrum of computational capability, from high-end domain controllers with multi-core processors and dedicated cryptographic co-processors down to simple body control modules, sensor nodes, and actuator controllers built around low-cost 32-bit microcontrollers with RAM measured in tens of kilobytes and no hardware cryptographic accelerator. On these constrained devices the computational cost of cryptographic operations becomes a real concern.

For such constrained ECUs, the cryptographic primitive must satisfy several requirements simultaneously: it must provide authenticated encryption with associated data (AEAD) to guarantee confidentiality, integrity, and authenticity in a single pass; it must execute within a time budget that does not interfere with real-time control tasks; and its implementation must fit within the available Flash and RAM without displacing application code. These requirements motivate a closer examination of whether AES-128-GCM (designed for general-purpose hardware) remains the most appropriate choice for every node in the vehicle network, or whether purpose-built lightweight alternatives deserve consideration.

1.3 The Limits of AES on Low-End Microcontrollers

AES-128-GCM is the dominant AEAD construction in modern security protocols and is well understood with respect to both security and implementation. On hardware that includes a dedicated AES accelerator, it executes efficiently and introduces negligible overhead. However, a large portion of automotive microcontrollers — particularly those in cost-sensitive, high-volume applications — are based on simple 32-bit cores such as the ARM Cortex-M0, which provide no hardware cryptographic acceleration. On these platforms, a software implementation of AES-128-GCM must perform the full AES key schedule and block cipher in software, and the GCM authentication step requires a 128-bit polynomial multiplication (GHASH) that maps poorly onto a minimal instruction set without hardware multiply-accumulate support. The result can be an implementation that is slower, larger in code size, or more energy-intensive than the available budget permits.

This raises a practical engineering question: for constrained ECUs where AES hardware acceleration is absent, is there a standardized alternative that offers better performance characteristics without sacrificing security?

1.4 The NIST Lightweight Cryptography Standardization Process

Recognizing the growing need for cryptographic primitives suited to constrained devices, NIST initiated a Lightweight Cryptography (LWC) standardization project in 2018 [2]. In February 2023, NIST announced ASCON as the winner [3]. The resulting standard was published in August 2025 as NIST SP 800-232 [4]. ASCON is a family of lightweight cryptographic algorithms based on a sponge construction, designed from the ground up for efficient software implementation on small processors. Its selection provides the embedded and automotive security community with a standardized, well-analyzed alternative to AES-GCM for use cases where resource constraints are the primary concern.

Despite this standardization, ASCON adoption in automotive and industrial embedded systems remains limited. AES continues to dominate deployed security stacks, in part due to inertia, existing hardware support, and a lack of quantitative performance data on representative automotive-class microcontrollers. This work aims to contribute directly to closing that gap.

1.5 Objective and Research Questions

This paper presents a quantitative benchmark comparing Ascon-AEAD128 and AES-128-GCM on two ARM Cortex-M microcontrollers representative of the low-to-mid range of the automotive ECU spectrum: the NUCLEO-F072RB (Cortex-M0, 48 MHz) and the NUCLEO-L476RG (Cortex-M4F, 80 MHz). Both boards serve as a simulation of the resource conditions found in constrained automotive ECUs. All measurements use software-only implementations to ensure a fair, hardware-accelerator-independent comparison applicable to the broadest range of real-world deployments.

The following research questions guide the study:

1. **Performance:** How do Ascon-AEAD128 and AES-128-GCM compare in execution time and throughput across a range of message sizes on each platform?
2. **Flash footprint:** What is the Flash code-size footprint of each implementation relative to the constraints of a typical automotive microcontroller?
3. **Optimization sensitivity:** How does compiler optimization level affect the relative performance of the two algorithms?
4. **Architecture dependence:** Does the performance gap between the two algorithms differ between the Cortex-M0 and the Cortex-M4F, and what does this imply for ECU selection?

1.6 Structure of This Paper

The remainder of this paper is organized as follows. Chapter 2 provides the technical background on AES-128-GCM, the ASCON family, and the target hardware platforms. Chapter 3 describes the benchmark methodology, including the measurement setup, timing approach, library selection, and build configuration. Chapter 4 presents the measurement results. Chapter 5 discusses the findings in the context of the research questions and draws practical conclusions for algorithm selection on constrained automotive hardware. Chapter 6 concludes the paper and outlines directions for future work.

2 Background

2.1 Authenticated Encryption with Associated Data (AEAD)

Cryptographic protocols for embedded communication commonly require three security objectives simultaneously: confidentiality (to prevent unauthorized reading of the message), integrity (to detect any modification during transmission), and authenticity (to verify that the message originates from a legitimate sender). Traditionally these properties were achieved by combining a separate cipher with a message authentication code (MAC), but the secure composition of two independent primitives is error-prone and incorrect combinations can introduce vulnerabilities even when each primitive is individually sound. Authenticated Encryption with Associated Data (AEAD) addresses this by combining all three objectives into a single, unified primitive.

An AEAD scheme takes four inputs: a secret key K , a nonce N (a value that must be unique per encryption under a given key), a plaintext message M , and a block of associated data AD . The associated data is authenticated but not encrypted: it remains readable in the clear while being bound to the ciphertext via the authentication tag. The scheme produces a ciphertext C and an authentication tag T . Decryption takes the same key and nonce, as well as the ciphertext, tag, and associated data, and returns the recovered plaintext only if the tag verifies successfully; otherwise it returns an explicit failure. This fail-closed behavior is essential in embedded contexts, where processing unauthenticated data before verification is a security risk.

For the constrained microcontroller setting considered in this work, two AEAD properties are of particular practical relevance. First, single-pass processing absorbs plaintext and produces the authentication tag in one sequential pass over the data, minimizing RAM requirements by avoiding the need to buffer the full message. Second, a compact state and key schedule reduce code size and initialization overhead, both of which are essential on devices with limited Flash. These properties will later serve as evaluation criteria when comparing AES-128-GCM and Ascon-AEAD128.

2.2 AES-128-GCM

2.2.1 The AES Block Cipher

The Advanced Encryption Standard (AES) is a substitution-permutation block cipher standardized by NIST in 2001 [1]. It operates on a fixed block size of 128 bits and supports key lengths of 128, 192, or 256 bits; this work uses the 128-bit key variant exclusively (AES-128). The cipher applies ten rounds of transformation to a 4×4 byte state matrix. Each round consists of four operations: `SubBytes` (a non-linear byte substitution via the AES S-box), `ShiftRows` (a cyclic shift of state rows), `MixColumns` (a linear diffusion step operating on state columns in $\text{GF}(2^8)$),

and AddRoundKey (XOR of the state with a round-specific subkey). The 128-bit key is expanded into eleven round keys before encryption begins, a process known as the key schedule.

The security of AES rests on two decades of public cryptanalysis, with no practical attack against the full cipher known. On a software-only Cortex-M0 implementation the MixColumns step and the S-box lookups impose a considerable cycle cost, because the M0 lacks the wider data path and lookup acceleration found on higher-end cores. This performance gap on minimal instruction-set architectures is the central motivation for evaluating a lightweight alternative.

2.2.2 Galois/Counter Mode (GCM)

AES alone is a block cipher and not an AEAD scheme: it encrypts exactly one 128-bit block and provides no authentication. Galois/Counter Mode (GCM) combines AES in counter mode (CTR) for encryption with a polynomial hash (GHASH) for authentication, yielding an AEAD construction [5].

In CTR mode, a counter value is encrypted by AES and the resulting keystream block is XORed with the plaintext. The counter is incremented for each successive block, which enables parallelized encryption independent of plaintext content. GHASH computes a 128-bit authentication tag using polynomial operations over the ciphertext blocks and associated data in $\text{GF}(2^{128})$, a finite field defined over 128-bit values. The hash key H is derived by encrypting a zero block under the session key at initialization.

GCM's parallelizability is its primary advantage on general-purpose hardware with wide data paths. Its principal weakness on constrained processors is the GHASH step: multiplication in $\text{GF}(2^{128})$ requires either a hardware carry-less multiply instruction or a software implementation using multiple 32-bit operations. The Cortex-M0 provides only a 32-bit multiply without a carry-less variant, so a software GHASH implementation on that core requires a series of XOR and shift operations that contribute noticeably to the total cycle count. GCM also requires a unique nonce per encryption, since nonce reuse under the same key is catastrophic.

2.2.3 Software-Only Scope

The STM32L476RG does not include a hardware AES peripheral; that block is present only in the STM32L486 and related parts of the same product family, as confirmed by the device reference manual. Both benchmark boards therefore present an identical software-only execution environment for this comparison. The absence of a hardware cryptographic accelerator on the selected boards is a deliberate design choice: restricting the comparison to software implementations throughout makes the results applicable to the broad class of constrained 32-bit microcontrollers that ship without vendor-specific AES acceleration, which represents the majority of cost-sensitive automotive nodes.

2 Background

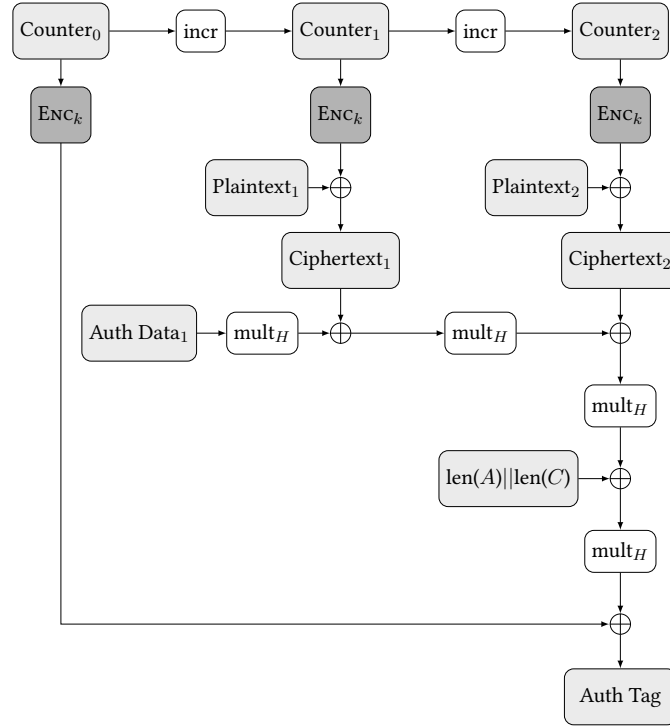


Figure 2.1: AES-128-GCM mode of operation. Adapted from a figure by D. Maimut in “TikZ for Cryptographers” [6] (CC0), following the GCM specification [5].

2.3 The ASCON Family

2.3.1 The ASCON Permutation

All ASCON variants use the same underlying permutation, written as p^a or p^b depending on how many rounds are applied. The permutation operates on a 320-bit state organized as five 64-bit words x_0 through x_4 . Each round applies three layers in sequence.

The first layer, p_C (constant addition), XORs a round-specific constant into word x_2 . The second layer, p_S (substitution), applies a 5-bit S-box in parallel to all 64 bit-sliced positions across the five state words. The third layer, p_L (linear diffusion), applies independent linear transformations to each of the five state words using rotations and XOR.

The full ASCON permutation uses twelve rounds (p^{12}) for initialization and finalization. The internal data-processing step uses eight rounds (p^8) for Ascon-AEAD128. On 32-bit microcontrollers the five 64-bit state words are held in pairs of 32-bit registers, so the rotation-and-XOR structure of the linear layer maps efficiently onto the available register file without memory traffic.

2.3.2 Sponge and Duplex Construction

ASCON uses a sponge-based construction (duplex variant) rather than the block-cipher-mode approach used by AES-GCM. The sponge model divides the state into two parts: the rate r (the part that interfaces with input and output) and the capacity c (the part that remains hidden from I/O, forming the security margin). For Ascon-AEAD128, the rate is 128 bits and the capacity is 192 bits, giving a total state of 320 bits.

In the duplex variant used for AEAD, plaintext is XORed into the rate portion of the state one rate-sized block at a time, followed by the permutation. The ciphertext is extracted from the state after the absorption, mixing encryption and authentication in a single sequential pass. This single-pass property means that memory buffering of the full message is never required, a significant advantage over constructions that must process the ciphertext twice (once for decryption, once for authentication).

The capacity portion of the state is never directly observable by an attacker and provides the security guarantee. For Ascon-AEAD128, the 192-bit capacity yields a claimed security level of 128 bits against generic attacks on both confidentiality and integrity.

2.3.3 Ascon-AEAD128

Ascon-AEAD128 is the primary AEAD member of the ASCON family and the algorithm selected as the winner of the NIST Lightweight Cryptography standardization process [3]. It is specified as a standard in NIST SP 800-232 [4]. It accepts a 128-bit key, a 128-bit nonce, associated data of arbitrary length, and a plaintext of arbitrary length, and produces a ciphertext of the same length as the plaintext together with a 128-bit authentication tag.

The processing pipeline proceeds in four phases. In the initialization phase, the 320-bit state is assembled from the key, nonce, and algorithm-specific initialization vector, then transformed by p^{12} . In the associated data phase, the associated data is absorbed into the rate in 128-bit blocks, each followed by p^8 ; the last block is padded, and a domain separation constant is XORed

into the state at the end of this phase. In the encryption phase, plaintext blocks are absorbed and ciphertext blocks are extracted in the same pass, each followed by p^8 . In the finalization phase, the key is XORed into the state, p^{12} is applied, and the authentication tag is extracted from the last 128 bits of the state.

For the benchmark in this work, Ascon-AEAD128 is evaluated using the official ASCON reference implementation in C [7].

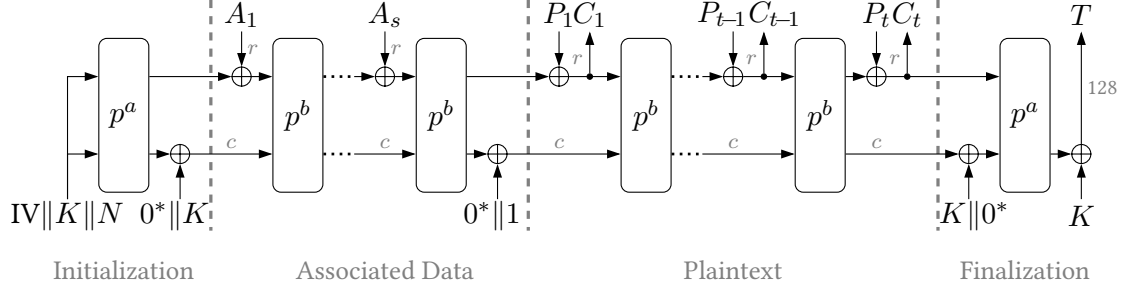


Figure 2.2: Ascon-AEAD128 mode of operation, from the Ascon specification [8]. In the diagram $a = 12$ for initialization and finalization, and $b = 8$ for data processing.

2.3.4 Ascon-Hash256

Ascon-Hash256 is the hash function member of the ASCON family. It produces a 256-bit digest using the same p^{12} permutation as Ascon-AEAD128, absorbing the input in 64-bit rate blocks and squeezing the digest in two 64-bit output blocks.

2.3.5 Ascon-80pq and the Post-Quantum Outlook

Ascon-80pq is a variant of Ascon-AEAD128 that extends the key length from 128 to 160 bits while keeping the nonce at 128 bits and the tag at 128 bits. The longer key is a precautionary measure against Grover’s algorithm, which effectively halves the security level of a symmetric key in bits against an adversary with a large-scale quantum computer. A 128-bit key offers approximately 64-bit post-quantum security under Grover’s model, while the 160-bit key of Ascon-80pq provides approximately 80-bit post-quantum security.

Ascon-80pq is part of the original ASCON v1.2 competition submission [8] and is not included in NIST SP 800-232, which standardizes only Ascon-AEAD128, Ascon-Hash256, Ascon-XOF128, and Ascon-CXOF128 [4].

2.4 Architectural Comparison: Sponge vs. Block Cipher + GCM

The structural differences between AES-128-GCM and Ascon-AEAD128 have concrete implications for software performance on constrained microcontrollers. Figures 2.1 and 2.2 show the two constructions schematically. AES-128-GCM is built from two largely independent components, the AES block cipher and the GHASH authenticator, each with its own state, and its key

schedule expands 128 bits into 176 bytes of round keys. Ascon-AEAD128 holds its entire state in the 320-bit permutation input, with no separate key schedule, and the permutation is built from bitwise rotations and XOR with no table lookups. Both algorithms process data in 128-bit units per core operation, so the throughput difference between them lies in the per-operation cost of the underlying primitive, which the benchmark measures.

2.5 Target Platforms

2.5.1 ARM Cortex-M0 — NUCLEO-F072RB

The NUCLEO-F072RB board is built around the STM32F072RB, an ARM Cortex-M0 core clocked at 48 MHz. It implements the ARMv6-M instruction set, with no DSP extensions, no floating-point unit, and no instruction or data caches, and it lacks a wide multiply-accumulate that would accelerate GHASH. Because the M0 has no Data Watchpoint and Trace (DWT) cycle counter, cycle-accurate measurement on this board uses TIM2, configured as a 32-bit free-running counter driven by the system clock.

2.5.2 ARM Cortex-M4F — NUCLEO-L476RG

The NUCLEO-L476RG board is built around the STM32L476RG, an ARM Cortex-M4F core clocked at up to 80 MHz. It implements the ARMv7E-M instruction set, which adds a single-cycle 32×32 -bit multiply producing a 64-bit result (UMULL and UMLAL, the variants relevant to GHASH), a single-precision floating-point unit (FPU), and an Adaptive Real-Time (ART) memory accelerator that reduces Flash access latency at high operating frequencies. For timing, the Cortex-M4F provides a DWT unit with the DWT->CYCCNT cycle counter, readable with a single memory-mapped register access.

2.6 Related Work

Performance comparisons between ASCON and AES-GCM on embedded hardware have appeared since ASCON's selection as the NIST LWC standard. Benchmarks on AVR 8-bit and MSP430 16-bit platforms demonstrated ASCON's advantage on ultra-constrained targets, but these results are not directly transferable to the 32-bit ARM Cortex-M deployments that represent the current generation of automotive microcontrollers. Figures for ARM Cortex-M targets have been published by the ASCON team and by independent researchers [7], but they often report a single message length rather than a throughput curve across a range of input sizes, which is needed to characterize behavior for the short messages typical in automotive CAN frames and sensor packets. To the authors' knowledge, no published work directly compares AES-128-GCM and Ascon-AEAD128 across both a Cortex-M0 and a Cortex-M4F with a controlled, software-only methodology that includes the compiler optimization level as an explicit variable.

3 Methodology

3.1 Overview and Experimental Design

This chapter describes how the benchmark was constructed and how every reported number was produced. The goal is a fair and reproducible comparison of Ascon-AEAD128 and AES-128-GCM under matched conditions on two constrained microcontrollers. Fairness here has a precise meaning: both algorithms run as software-only implementations, both are measured through an identical timing harness, both are compiled with the same toolchain and the same set of optimization levels, and the one-time setup cost is excluded from the timed region for both in the same way.

The experimental design varies four factors: the algorithm (Ascon-AEAD128 or AES-128-GCM), the hardware platform (Cortex-M0 or Cortex-M4F), the input message size (swept across several orders of magnitude), and the compiler optimization level (swept from no optimization through size optimization). Each combination is measured many times and reduced to summary statistics offline. Code size is measured separately, as a static property of the compiled binaries rather than a runtime measurement.

3.2 Target Platforms

Two STMicroelectronics Nucleo-64 development boards represent the low and middle of the constrained range. Their relevant properties are summarized in Table 3.1. Both boards expose an on-board ST-LINK debugger with a USB virtual COM port routed to USART2, which carries the measurement output.

Table 3.1: Target platform properties

Property	NUCLEO-F072RB	NUCLEO-L476RG
Microcontroller	STM32F072RBT6	STM32L476RGT6
Core	Cortex-M0	Cortex-M4F
Maximum clock	48 MHz	80 MHz
Flash	128 KB	1 MB
SRAM	16 KB	128 KB
Hardware AES	None	None
Cycle counter (DWT)	Absent	Present
Instruction prefetch	None	ART accelerator
Floating-point unit	None	Single precision

3.2.1 NUCLEO-F072RB (Cortex-M0)

The F072 carries a Cortex-M0 core with no hardware multiplier for wide operands, no instruction cache, no branch predictor, and no floating-point unit. It represents the lower end of the device range, where every cycle and every kilobyte is scarce. Its 16 KB of SRAM is the binding constraint on this study, and it limits the largest message size that can be benchmarked on this board.

3.2.2 NUCLEO-L476RG (Cortex-M4F)

The L476 carries a Cortex-M4F core with a single-cycle hardware multiplier, a single-precision floating-point unit, and the ST ART accelerator, which prefetches and caches instructions from Flash. It represents the middle of the range. The L476 has substantially more memory than the F072, so memory is not a constraint on this board, but it is included to expose how the relative performance of the two algorithms depends on the processor architecture.

The specific part on the NUCLEO-L476RG does not contain a hardware AES block; only the L486 and related parts in the same family include one. This absence is useful for the study, because it means there is no hardware cryptographic path that could be engaged accidentally, so the software comparison is consistent on both boards.

3.2.3 Clocking and the Matched-Frequency Decision

Both boards run at 48 MHz for the timing comparison. The L476 is capable of 80 MHz, so it is intentionally underclocked to the F072 ceiling. This choice removes clock frequency as a variable: any difference in measured time between the two boards at the same clock then reflects the processor architecture alone, not a difference in operating frequency.

The measurements are reported primarily as cycle counts rather than wall-clock time, since the timer increments once per processor clock (see Section 3.6), so one timer tick equals one clock cycle and the cycle count is independent of the operating frequency.

3.3 Toolchain and Build System

All firmware was built with a single toolchain to keep the comparison consistent. The compiler is `arm-none-eabi-gcc` version 14.3.1, supplied with the STM32CubeCLT bundle. The build is driven by CMake with the Ninja generator, both bundled with CubeCLT. Project skeletons, including peripheral initialization and the linker script, were generated with STM32CubeMX configured for CMake output. The host operating system was Windows.

The base compiler flags follow the CubeMX template. They include `-ffunction-sections`, `-fdata-sections`, and linker garbage collection (`-Wl, -gc-sections`), the size-oriented newlib-nano C library (`-specs=nano.specs`), and the architecture flags appropriate to each core. On the L476 these include the hardware floating-point ABI, although floating-point arithmetic is not used by either algorithm.

The optimization level is the one build variable that changes across the measurement matrix. It is controlled through a single CMake cache variable set per build preset. Each preset reconfigures the build at one optimization level, from `-O0` through `-O5`, so that the same source produces a family of binaries that differ only in optimization.

3.4 Cryptographic Implementations

Both implementations are taken from existing open-source projects rather than written for this study. This keeps the comparison representative of what a developer would actually deploy, and it avoids the risk of an unrepresentative hand-written implementation favoring one side.

3.4.1 AES-128-GCM (mbedTLS)

AES-128-GCM is provided by mbedTLS version 3.6.2 [9]. mbedTLS was selected because it is a widely deployed, well-maintained library with a complete and verified AES-GCM implementation, which lends the comparison credibility. The library was reduced to a minimal configuration containing only the modules required for AES-128-GCM.

Three configuration choices affect the measurements and are disclosed here, because the reported numbers are only interpretable alongside them.

First, the small four-bit GHASH multiplication table is used. The larger eight-bit table is faster per byte but enlarges the GCM context by roughly 3.8 KB of RAM. The small table was chosen because it represents a genuinely lightweight configuration of AES-GCM, matching the constrained-device framing of this work. As a consequence, the reported AES-GCM throughput represents the memory-frugal end of what mbedTLS can achieve rather than its speed ceiling. This point is revisited in the discussion.

Second, the AES lookup tables are placed in Flash rather than generated in RAM at initialization. This shifts roughly 8 KB from RAM to Flash, which is the realistic trade-off on a part with only 16 KB of SRAM. It affects the code-size results but has no meaningful effect on runtime.

Third, only the 128-bit key length is compiled. The 192-bit and 256-bit key schedules are removed. Because only AES-128 is benchmarked, this has no effect on runtime and only a small effect on Flash size.

3.4.2 Ascon-AEAD128 (Reference Implementation)

Ascon-AEAD128 is provided by the official ASCON reference implementation in C [7]. The reference variant was chosen over the optimized variants for portability and because it corresponds directly to the standardized specification. The implementation uses a 16-byte key, a 16-byte nonce, and produces a 16-byte authentication tag, with a rate of 16 bytes.

The choice of the reference variant has a visible effect on code size at high optimization levels. This is reported and discussed rather than hidden, because it reflects what a developer integrating the standard reference code would observe.

3.4.3 Uniform Wrapper Interface

Each implementation is reached through a thin wrapper with a matching signature, so that the benchmark harness calls both algorithms the same way. The AES wrapper separates key setup from encryption, so that the one-time key schedule can be hoisted out of the timed region. The ASCON single-pass AEAD interface has no separable key schedule, so its encryption call is timed in full. Neither wrapper performs additional copying inside the timed path beyond what the underlying library requires.

3.5 Correctness Verification

No timing measurement was trusted until the implementation producing it was shown to be correct. Each algorithm was checked against a published Known Answer Test vector, the NIST GCM vector for AES-128-GCM and a Lightweight Cryptography Known Answer Test vector for Ascon-AEAD128, confirming both the ciphertext and the authentication tag. Each was additionally checked with a round-trip test, where decryption returns the original plaintext, and a forgery-rejection test, where a modified tag is correctly rejected. Both verification routines run on the target hardware at startup, before the benchmark begins, print a pass result over the serial link, and halt on failure so that no subsequent timing for that build is interpreted.

3.6 Measurement Methodology

Both boards use the TIM2 timer as the timing source. TIM2 is a 32-bit timer on both families, configured identically on each board as a free-running up-counter driven by the internal clock with the prescaler set to zero, so the counter increments once per processor clock cycle and one tick equals one cycle. At 48 MHz the 32-bit counter wraps approximately every 89 seconds, far longer than any single measurement, and all elapsed-time computations use unsigned 32-bit subtraction, which handles a single wrap during a measurement correctly. The Cortex-M0 on the F072 does not provide the Data Watchpoint and Trace cycle counter available on the Cortex-M4F, so using TIM2 on both boards gives a comparison between identically configured peripherals rather than between two architecturally different timing mechanisms.

The benchmark macro disables all maskable interrupts for the duration of the measured region and re-enables them afterward, which prevents the periodic system tick interrupt and any peripheral interrupts from polluting short measurements. The effect was confirmed during validation, where masking reduced the per-iteration variation from roughly 28 ticks to zero on a reference workload. Each measurement also runs the workload once before the timed iterations begin and discards the result. This warmup primes the L476 ART accelerator and any related microarchitectural state, on which the first run differs from the steady state by roughly 1.5% at small sizes. On the F072 there is no measurable warmup effect, because the Cortex-M0 has no instruction cache or branch predictor that could be cold.

At higher optimization levels the compiler can remove a computation whose result is never used. To prevent this, each workload writes one byte of its output into a `volatile` variable, which forces the compiler to produce the output because it cannot prove the write is unnecessary. This protection is verified indirectly by checking that the smallest input size still records a non-zero time at every optimization level.

The timed region contains only the authenticated-encryption operation. For AES-128-GCM the key schedule is performed once before the measurements begin and is not included. For Ascon-AEAD128 the single-pass interface performs key setup as part of the encryption call and cannot separate it, so the full call is timed. This treats the one-time setup symmetrically: the repeated per-call cost is measured for both, and the fixed setup that a real deployment would perform once is excluded from both where the interface allows. All measurements encrypt plaintext with no associated data.

Each measured iteration is streamed over the serial link as a row of comma-separated values containing the algorithm, the board, the clock frequency, the input size, the iteration index, and the measured ticks. Summary statistics, namely the median, minimum, maximum, and interquartile range, are computed offline rather than on the device, which keeps the on-device harness minimal and allows the analysis to be revised without re-running the measurements. Under interrupt masking the per-iteration variation was zero on both boards for all measured workloads, so the median, minimum, and maximum coincide.

3.7 Benchmark Parameters

3.7.1 Input Sizes

Message sizes are swept on a logarithmic scale. On the L476 the sizes are 1, 10, 100, 1000, and 10 000 bytes. On the F072 the largest size is omitted, because the buffers required for a 10 000-byte message do not fit within 16 KB of SRAM, so the F072 sizes are 1, 10, 100, and 1000 bytes. The logarithmic spread separates the fixed per-call cost, which dominates at small sizes, from the per-byte cost, which dominates at large sizes.

Each size is measured for 100 iterations after the discarded warmup run.

3.7.2 Optimization-Level Matrix

Each algorithm on each board is compiled at five optimization levels: -O0, -O1, -O2, -O3, and -Os. This sweep shows how much of the measured performance depends on optimization rather than on the algorithm itself, and it identifies the level at which each algorithm performs best, which differs by algorithm and by architecture. Reporting only a single optimization level would hide both effects.

3.8 Footprint Measurement

Code size is measured as a static property of the compiled binaries using `arm-none-eabi-size`, which reports the size of the program text, the initialized data, and the zero-initialized data sections. Flash usage is taken as the sum of the text and initialized-data sections.

Because a single binary links both algorithms together with the common platform and harness code, the size of one combined binary cannot attribute code to one algorithm or the other. To obtain a per-algorithm figure, three build variants are produced from the same source through a compile-time selection: a baseline variant (platform and harness code, no cryptographic implementation), an AES variant (adds AES-128-GCM), and an ASCON variant (adds Ascon-AEAD128). The marginal cost of each algorithm is the difference between its variant and the baseline.

Footprint is reported primarily at the size-optimization level, because that is the configuration a memory-constrained deployment would use. Footprint at the other optimization levels is also recorded, as the code size of the ASCON reference implementation varies considerably with optimization.

3.9 Harness Validation

Before any cryptographic code was measured, the harness was validated with a memcpy workload of known behavior, measured at sizes 1, 10, 100, and 1000 bytes for 100 iterations each. Four checks were defined: the measured time should grow roughly in proportion to the input size; the per-iteration variation should be at most a few ticks; the smallest input size should record a non-zero time; and the reported clock frequency should read 48 MHz on both boards.

The results of this validation are reported in Section 4.2.

3.10 Reproducibility and Known Limitations

The exact mbedTLS configuration is committed to the repository, and the commit hashes of mbedTLS and the ASCON reference implementation are recorded [9, 7]. The compiler, CMake, Ninja, and CubeCLT versions are recorded, as are the board hardware revisions and the ST-LINK firmware versions.

Several limitations are disclosed for honesty of interpretation. The reported times include a small fixed overhead from the timer read pair, the loop branch, and the volatile store. At the largest size this overhead is on the order of one percent of the total; at the smallest size it dominates, so the smallest-size figures should be read as a measurement of fixed per-call cost rather than of encryption work. The harness does not subtract this overhead, and raw ticks are reported as measured. The AES-GCM measurements use the small GHASH table, as disclosed in Section 3.4.1. The ART accelerator is left enabled on the L476, which is the realistic deployment configuration, and cycle counts are reported so that the comparison does not depend on its effect.

4 Results

4.1 Overview

This chapter presents the measured results in three parts. Section 4.2 reports the harness validation, which establishes that the timing infrastructure is trustworthy. Section 4.3 reports the execution-time results across message sizes and optimization levels. Section 4.4 reports the throughput results in cycles per byte. Section 4.5 reports the code-size results. All timing figures are cycle counts at 48 MHz, where one timer tick equals one processor cycle. All results derive from the single consolidated data set, and every figure and table is reproducible from it.

4.2 Harness Validation

The memcpy validation passed all four checks on both boards. The measured time grew in proportion to the input size, the per-iteration variation was zero after the discarded warmup run, the smallest input size recorded a non-zero time, and both boards reported the expected 48 MHz clock. The per-iteration determinism observed here persisted through all later measurements: across the cryptographic benchmarks the median, minimum, and maximum coincided for every measured point, so the timing results are reported as single median values without dispersion.

The ratio between the two boards for the memcpy workload settled at approximately 2.8 to 1 at the largest size, with the Cortex-M0 slower than the Cortex-M4F at the same clock. This figure is the architectural baseline against which the cryptographic ratios in the following sections are read.

4.3 Execution Time

4.3.1 Runtime Across Message Sizes

Figure 4.1 shows median execution time against input size on logarithmic axes, with one panel per board, at optimization level -O2. On both boards Ascon-AEAD128 is faster than AES-128-GCM at every measured size. The curves share a characteristic shape: at the smallest sizes the time is nearly constant, because the fixed per-call cost dominates; above approximately 100 bytes the time rises in proportion to the input size, which is the per-byte encryption cost. The vertical separation between the two algorithms is present across the whole range and is wider on the Cortex-M0 board than on the Cortex-M4F board.

4.3.2 Effect of Optimization Level

Table 4.1 gives the median execution time at 1000 bytes across the five optimization levels.

4 Results

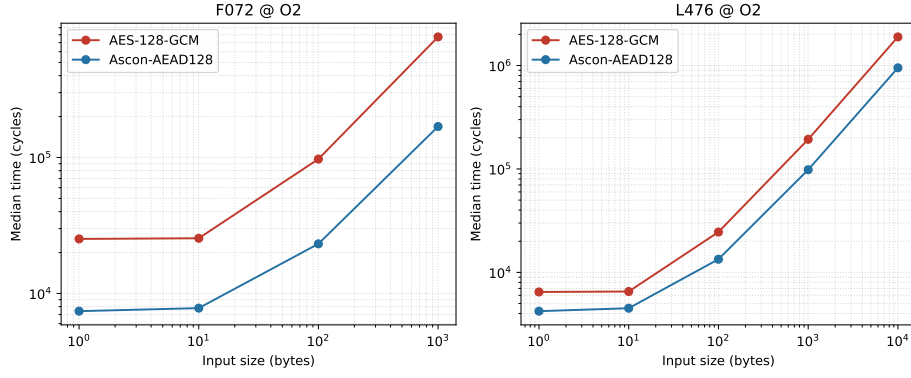


Figure 4.1: Median execution time vs. message size at -02 on both boards (logarithmic axes). Ascon-AEAD128 is consistently faster than AES-128-GCM at every size.

The dominant effect is the step from -00 to -01. It improves both algorithms but unevenly: on the Cortex-M0 it reduces the Ascon-AEAD128 time at 1000 bytes by a factor of about eight and the AES-128-GCM time by a factor of about three. Between -01, -02, and -03 the changes are smaller and not uniform across the two boards. On the Cortex-M4F, AES-128-GCM gains nothing from -03 over -02 while Ascon-AEAD128 continues to improve; on the Cortex-M0 the pattern is reversed. At -0s both algorithms are slower than at -02 on both boards.

Table 4.1: Median execution time (cycles) and ratio at 1000 bytes per optimization level.

Level	M0 AES	M0 ASCON	M0 ratio	M4 AES	M4 ASCON	M4 ratio
-00	2,661,457	1,387,063	1.92	845,537	726,789	1.16
-01	792,911	170,741	4.64	242,980	104,422	2.33
-02	770,536	168,921	4.56	193,311	98,262	1.97
-03	485,227	145,888	3.33	193,283	70,194	2.75
-0s	787,284	268,899	2.93	269,250	139,993	1.92

At -00 the two algorithms are close, with a ratio of 1.16 on the Cortex-M4F. Once optimization is enabled the ratio is larger, and at every optimized level it is greater on the Cortex-M0 than on the Cortex-M4F. The largest ratio in favor of Ascon-AEAD128 is about 4.6, at -01 and -02 on the Cortex-M0.

4.4 Throughput

Figure 4.2 shows median cycles per byte against input size on logarithmic axes, with one panel per board, at -02. At the smallest size the cost per byte is highest, because the fixed per-call cost is divided across a single byte. As the message grows the cost per byte falls toward a flat asymptote, which is the per-byte encryption rate.

In the large-message region the per-byte rates at -02 are as follows. On the Cortex-M0 at

4 Results

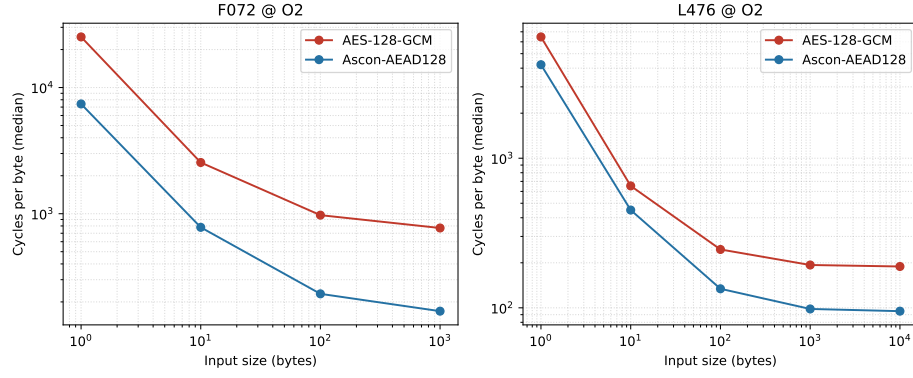


Figure 4.2: Median cycles per byte vs. message size at -O2 on both boards (logarithmic axes). The asymptotic value at large sizes reflects the steady-state per-byte encryption rate.

1000 bytes, Ascon-AEAD128 reaches approximately 169 cycles per byte and AES-128-GCM approximately 771 cycles per byte. On the Cortex-M4F at 10,000 bytes, Ascon-AEAD128 reaches approximately 95 cycles per byte and AES-128-GCM approximately 189 cycles per byte.

At the smallest message size of one byte, where the figure reflects the fixed per-call cost, the values at -O2 are 7,410 cycles for Ascon-AEAD128 and 25,185 cycles for AES-128-GCM on the Cortex-M0, and 4,219 cycles for Ascon-AEAD128 and 6,470 cycles for AES-128-GCM on the Cortex-M4F.

4.5 Code Size

Figure 4.3 shows the marginal Flash cost of each algorithm at -Os for both boards. Table 4.2 gives the marginal Flash cost at every optimization level.

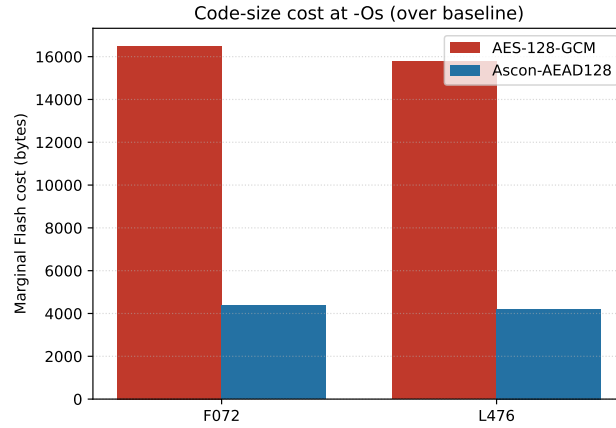


Figure 4.3: Marginal Flash cost at -Os for each algorithm and board. Ascon-AEAD128 requires approximately $3.8\times$ less Flash than AES-128-GCM at this level.

4 Results

At -0s, Ascon-AEAD128 adds 4,376 bytes of Flash on the Cortex-M0 and 4,208 bytes on the Cortex-M4F. AES-128-GCM adds 16,500 bytes on the Cortex-M0 and 15,776 bytes on the Cortex-M4F. The ratio of AES-128-GCM to Ascon-AEAD128 marginal Flash is approximately 3.8 on both boards.

Table 4.2: Marginal Flash cost (bytes) and ratio per optimization level.

Level	M0 AES	M0 ASCON	M0 ratio	M4 AES	M4 ASCON	M4 ratio
-00	26,296	12,836	2.05	23,256	6,776	3.43
-01	17,396	27,924	0.62	16,496	24,208	0.68
-02	17,476	27,504	0.64	16,584	23,264	0.71
-03	24,208	43,024	0.56	17,720	34,560	0.51
-0s	16,500	4,376	3.77	15,776	4,208	3.75

The Ascon-AEAD128 marginal Flash cost varies strongly with optimization level, as Table 4.2 shows. At -0s it is the smaller of the two algorithms by a wide margin. At -01, -02, and -03 it is larger than the AES-128-GCM cost, reaching a maximum at -03 of 43,024 bytes on the Cortex-M0 and 34,560 bytes on the Cortex-M4F. The AES-128-GCM marginal Flash cost is comparatively stable across levels, between roughly 16,000 and 26,000 bytes. The shape of the Ascon-AEAD128 cost is the same on both boards, rising from -00 through a peak at -03 and then falling sharply at -0s.

4.6 Summary of Results

Across both boards and all message sizes, Ascon-AEAD128 executed faster than AES-128-GCM at every optimization level. The advantage was smallest at -00 and larger once optimization was enabled. At equal clock frequency the advantage was consistently larger on the Cortex-M0 than on the Cortex-M4F, reaching a factor of about 4.6 at 1000 bytes on the Cortex-M0. In code size, Ascon-AEAD128 was substantially smaller than AES-128-GCM at -0s, by a factor of about 3.8 on both boards, while at the speed-oriented optimization levels its reference implementation was larger than AES-128-GCM due to the expansion of the permutation code. These results are interpreted in relation to the four research questions in Chapter 5.

5 Discussion

5.1 Performance (Research Question 1)

The first research question asks how the two algorithms compare in execution time and throughput across message sizes on each platform. The measurements give a clear answer: Ascon-AEAD128 was faster than AES-128-GCM at every measured message size, on both boards, and at every optimization level. The advantage was present in both the fixed per-call cost that dominates short messages (Section 4.3) and the per-byte cost that dominates long messages (Section 4.4). Ascon-AEAD128 leads in both regimes, so the conclusion does not depend on the message-size profile of the application.

The reason for the difference lies in the structure of the two algorithms. AES in software relies on table lookups for the substitution step, and the GCM authentication step performs a multiplication in a binary field, built from many smaller operations when no wide hardware multiplier is available. Ascon-AEAD128 is built from a permutation of bitwise operations, exclusive-or, bitwise-and, and rotation, applied to a small state held in registers. These operations are native to both cores and require no table memory. The single-pass duplex construction also avoids the separate encryption and authentication passes that AES-128-GCM performs. The measured advantage is the practical consequence of this design difference.

5.2 Flash Footprint (Research Question 2)

The second research question asks about the Flash footprint of each implementation relative to the constraints of a small microcontroller. The answer depends on the optimization level, and the size-optimized level is the one relevant to a memory-constrained deployment. At -Os, AES-128-GCM required approximately 3.8 times the code space of Ascon-AEAD128, consistently on both boards (Section 4.5). The small code size of Ascon-AEAD128 follows from its single permutation and its lack of precomputed lookup tables, whereas AES-128-GCM brings the AES tables and the field-multiplication code.

This advantage at -Os does not hold at the speed-oriented optimization levels, for the reason examined in Section 5.3. The footprint result is therefore reported at the optimization level that a constrained deployment would actually choose, where Ascon-AEAD128 is both smaller and, from the timing results, faster.

5.3 Optimization Sensitivity (Research Question 3)

The third research question asks how the compiler optimization level affects the relative performance of the two algorithms. The measurements show that the optimization level matters a

great deal, in two distinct ways.

The first concerns execution time. The largest single improvement for both algorithms came from enabling optimization at all — the step from -00 to -01. At 1000 bytes on the Cortex-M0 this step reduced the Ascon-AEAD128 time by a factor of about eight and the AES-128-GCM time by a factor of about three. This uneven response means the relative advantage of Ascon-AEAD128 is understated by any measurement taken at -00. A study that benchmarked only unoptimized code would have reported the two algorithms as close — with a ratio of 1.16 on the Cortex-M4F — and would have missed the larger advantage that appears under the optimization a real build would use. The choice to sweep the optimization level, rather than report a single level, is justified by this observation.

The behavior at the higher levels was not uniform. On the Cortex-M4F, AES-128-GCM gained nothing from -03 over -02, consistent with the common observation that aggressive optimization yields little for table-driven code on these cores, while Ascon-AEAD128 continued to improve at -03. On the Cortex-M0 the pattern was reversed, with AES-128-GCM improving substantially at -03 and Ascon-AEAD128 improving only modestly. The -0s level was slower than -02 for both algorithms on both boards, because size optimization disables the inlining and loop unrolling that the cryptographic inner loops benefit from.

The second way the optimization level matters concerns code size, and it is the more striking effect. The Flash cost of the Ascon-AEAD128 reference implementation varied strongly with the optimization level, while the AES-128-GCM cost was comparatively stable. The Ascon-AEAD128 cost rose from -00 to a peak at -03, reaching 43,024 bytes on the Cortex-M0 and 34,560 bytes on the Cortex-M4F, then fell sharply at -0s to roughly 4 KB. The cause is the unrolling of the permutation: at the speed-oriented levels the compiler unrolls the permutation rounds into straight-line code, which the authenticated-encryption routine includes at several call sites, expanding the binary. At -0s the compiler keeps the rounds rolled, and the code collapses to a small size. The same shape appeared on both boards, showing that the effect is a property of the implementation and the compiler rather than of the processor.

This links the second and third research questions. The footprint advantage of Ascon-AEAD128 is real but conditional on the optimization level: it exists at -0s and is reversed at the speed-oriented levels. The advantage and this condition should be reported together.

5.4 Architecture Dependence (Research Question 4)

The fourth research question asks whether the performance gap between the two algorithms differs between the Cortex-M0 and the Cortex-M4F, and what this implies for device selection. The measurements show that the gap is consistently larger on the weaker core.

At equal clock frequency and at 1000 bytes, the ratio of AES-128-GCM time to Ascon-AEAD128 time was about 4.6 at -01 and -02 on the Cortex-M0, against about 2.0 at the same levels on the Cortex-M4F. At every optimized level the ratio was larger on the Cortex-M0. The advantage of Ascon-AEAD128 therefore grows as the processor becomes more constrained.

This can be read against the architectural baseline established in the harness validation. For the plain memory workload the Cortex-M0 was about 2.8 times slower than the Cortex-M4F at the same clock. If both algorithms simply scaled with this architectural factor, their ratio to each

other would be the same on both boards. The fact that the AES-128-GCM disadvantage grows by more than this factor on the Cortex-M0 indicates that AES-128-GCM contains operations that are especially costly on the weaker core. The field multiplication in the GCM authentication step is the principal example: it must be synthesized from many small operations when no wide hardware multiplier is present, which is the case on the Cortex-M0. Ascon-AEAD128, built from operations the Cortex-M0 executes natively, stays closer to the architectural baseline.

The implication for device selection is direct: the more constrained the processor, the stronger the case for Ascon-AEAD128 over AES-128-GCM. On the lowest-end cores, which are also the most numerous and the most cost-sensitive, the advantage is largest.

5.5 The Fairness of the Comparison

The comparison was designed to avoid handicapping either algorithm, and one configuration choice deserves explicit defense. AES-128-GCM was measured with the small field-multiplication table, the memory-frugal configuration disclosed in Section 3.4.1. The larger table would improve throughput at a RAM cost that Ascon-AEAD128 does not require, and on a part with 16 KB of SRAM that is exactly the kind of cost a constrained deployment may decline.

The reported AES-128-GCM figures therefore represent the memory-frugal end of AES-128-GCM rather than its fastest possible configuration. Enabling the larger table would narrow the throughput gap at large messages, at a RAM cost Ascon-AEAD128 does not pay, and would not change the code-size or small-message conclusions. The comparison is thus a fair one between two lightweight configurations, and the configuration choice is named and justified so the result cannot be read as a weakened AES-128-GCM.

5.6 Practical Conclusions

The results support a consistent practical recommendation for the class of devices studied. For authenticated encryption on a constrained microcontroller without hardware AES acceleration, Ascon-AEAD128 is the stronger choice on the measured criteria: it was faster than AES-128-GCM at every message size and optimization level, smaller in code at the size-optimized level that such a device would use, and its advantage was largest on the most constrained processor.

Two qualifications accompany this recommendation. First, the speed advantage and the size advantage do not occur at the same optimization level for the reference implementation. The size advantage holds at -0s, where Ascon-AEAD128 is both smaller and faster than AES-128-GCM, so a deployment that builds for size obtains both benefits at once. A deployment that builds for speed at -02 or -03 obtains the speed advantage but accepts a larger Ascon-AEAD128 code size due to the unrolling of the permutation. Second, these conclusions concern software-only implementations. On a device with a hardware AES accelerator the balance would differ, and that case was deliberately outside the scope of this work in order to isolate the software comparison.

Within those qualifications, the measurements give a quantitative basis for preferring Ascon-AEAD128 on constrained, software-only targets, and they quantify how the advantage grows as the device becomes more limited.

6 Conclusion and Outlook

6.1 Conclusion

This work presented a measurement-based comparison of Ascon-AEAD128 and AES-128-GCM on a Cortex-M0 and a Cortex-M4F, using software-only implementations under matched conditions. It was motivated by the absence of quantitative performance data for the standardized lightweight algorithm on representative constrained hardware, and it set out to provide that data across execution time, code size, optimization level, and processor architecture.

The measurements answer the four research questions consistently: Ascon-AEAD128 was faster at every message size and optimization level on both boards, required about one quarter of the Flash of AES-128-GCM at the size-optimized level, showed an advantage that was understated at -O0 and clearer once optimization was enabled, and showed an advantage that grew on the more constrained Cortex-M0. Taken together the results give a quantitative basis for preferring Ascon-AEAD128 on constrained, software-only targets, subject to the qualification that the speed advantage and the code-size advantage are obtained together only when building for size.

6.2 Outlook

Several directions extend this work. First, measuring the larger-table AES-128-GCM configuration would bound the fastest AES-128-GCM throughput achievable on these boards and quantify the speed gained for the additional RAM. Second, running the Cortex-M4F at its native 80 MHz would give wall-clock figures for that board at full speed, confirming the cycle-count results that are independent of operating frequency. Third, comparing Ascon-Hash256 against SHA-256 on the same hardware would extend the comparison to the hashing use case that arises in key derivation and integrity checking. Fourth, repeating the study on a part with a hardware AES accelerator would characterize how that shifts the balance and would mark the boundary of the recommendation made here. Fifth, measuring Ascon-80pq, which shares the same permutation as Ascon-AEAD128 [8], would quantify the cost of its longer key and position the comparison against longer-term security requirements. These directions all build on the measurement infrastructure established here, which was validated, frozen, and documented for reproducibility.

Bibliography

- [1] National Institute of Standards and Technology, “Advanced Encryption Standard (AES),” National Institute of Standards and Technology, Federal Information Processing Standard 197, 2001.
- [2] —, “Submission Requirements and Evaluation Criteria for the Lightweight Cryptography Standardization Process,” National Institute of Standards and Technology, Tech. Rep., 2018, available at <https://csrc.nist.gov/projects/lightweight-cryptography>.
- [3] —, “NIST Selects ‘Ascon’ for Lightweight Cryptography Standardization,” NIST Announcement, February 2023, 2023, <https://www.nist.gov/news-events/news/2023/02/nist-selects-ascon-lightweight-cryptography-standardization>.
- [4] M. Sönmez Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, “Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions,” National Institute of Standards and Technology, NIST Special Publication 800-232, Aug. 2025.
- [5] M. J. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” National Institute of Standards and Technology, NIST Special Publication 800-38D, 2007.
- [6] J. Jean, “TikZ for cryptographers,” <https://www.iacr.org/authors/tikz/>, 2016.
- [7] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, “ASCON reference software (ascon-c),” GitHub, 2023, <https://github.com/ascon/ascon-c>, commit 446347f21b209f3921c65ece70027c366cbe1693.
- [8] —, “ASCON v1.2,” Submission to the NIST Lightweight Cryptography Standardization Project, 2021, <https://ascon.iaik.tugraz.at/>.
- [9] Mbed TLS Contributors, “Mbed TLS,” GitHub, 2024, <https://github.com/Mbed-TLS/mbedtls>, commit 107ea89daaefb9867ea9121002fbbdf926780e98.